SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

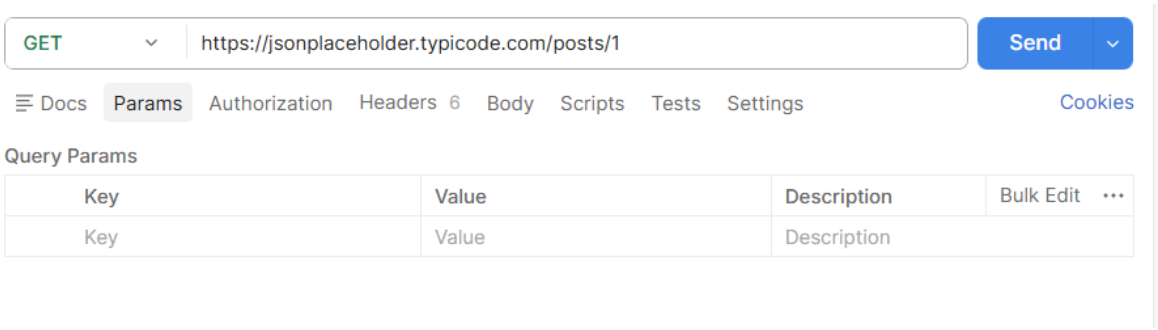
Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

Before you start: *you only need a browser and Postman. Base URL for every task: https://jsonplaceholder.typicode.com — no signup or API key required.*

Tasks

TASK 1 Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.



Output:

b) Record the status code you get back.

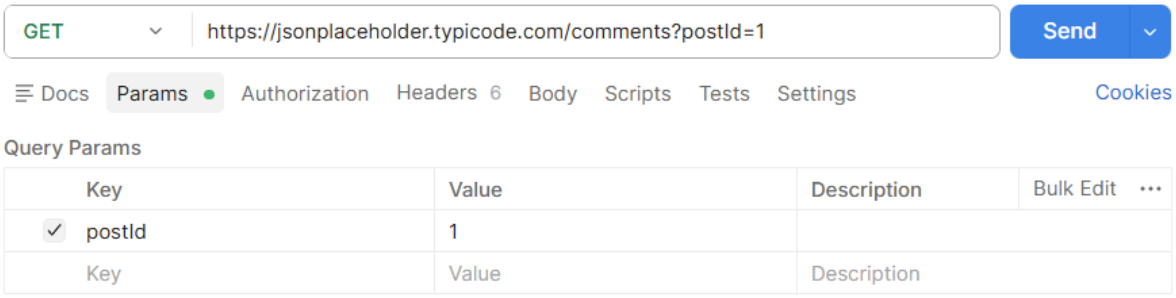
Output:

c) In one sentence, explain why this request works even with nothing in the Headers tab.

The request works without headers because this GET request only retrieves public data and does not require authentication or additional information.

TASK 2 Filtering With Query Parameters

a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.



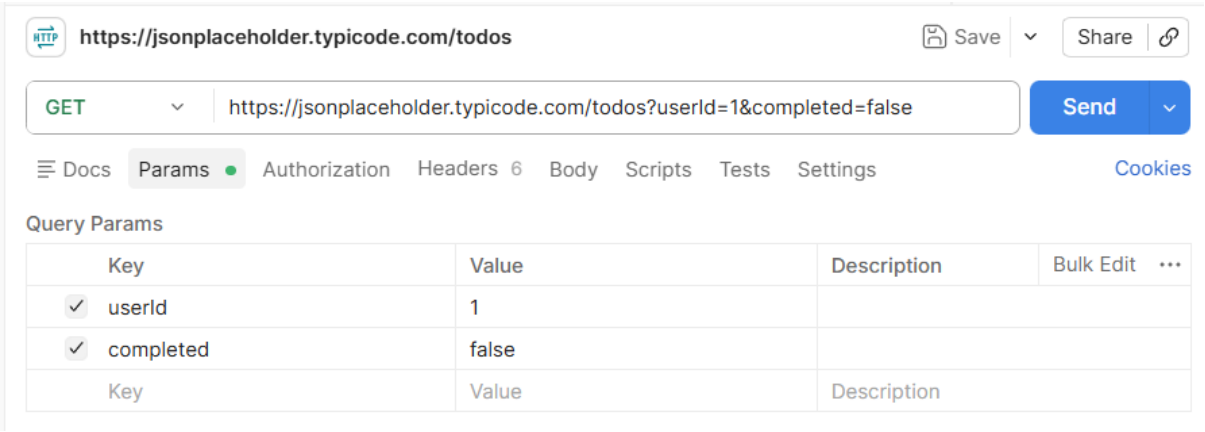
Output:


```
{
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam
        eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et
        nam sapiente accusantium"
},
{
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis
        occaecati quod ullam at\nvoluptatem error expedita pariatur\nnihil sint
        nostrum voluptatem reiciendis et"
},
{
    "postId": 1,
    "id": 3,
    "name": "odio adipisci rerum aut animi",
    "email": "Nikita@garfield.biz",
    "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita
        occaecati aliquam eveniet laudantium\nomnis quibusdam delectus saepe
        quia accusamus maiores nam est\ncum et ducimus et vero voluptates
        excepturi deleniti ratione"
},
{
    "postId": 1,
    "id": 4,
    "name": "alias odio sit",
    "email": "Lew@alysha.tv",
    "body": "non et atque\noccaecati deserunt quas accusantium unde odit nobis
        qui voluptatem\nquia voluptas consequuntur itaque dolor\net qui rerum
        deleniti ut occaecati"
},
{
    "postId": 1,
    "id": 5,
    "name": "vero eaque aliquid doloribus et culpa",
    "email": "Hayden@althea.biz",
    "body": "harum non quasi et ratione\ntempore iure ex voluptates in
        ratione\nharum architecto fugit inventore cupiditate\nvoluptates magni
        quo et"
}
```

b) Confirm the URL automatically updates to include ?postId=1.

Yes the postman automatically updated the URL to include ?postID=1 when the query parameter and value was added.

c) Now combine two filters at once on /todos: userId = 1 and completed = false.



Output:


```
{
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
},
{
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
},
{
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
},
{
    "userId": 1,
    "id": 5,
    "title": "laboriosam mollitia et enim quasi adipisci quia provident illum",
    "completed": false
},
{
    "userId": 1,
    "id": 6,
    "title": "qui ullam ratione quibusdam voluptatem quia omnis",
    "completed": false
},
{
    "userId": 1,
    "id": 7,
    "title": "illo expedita consequatur quia in",
    "completed": false
},
{
    "userId": 1,
    "id": 9,
    "title": "molestiae perspiciatis ipsa",
    "completed": false
},
{
    "userId": 1,
    "id": 13,
    "title": "et doloremque nulla",
    "completed": false
},
{
    "userId": 1,
    "id": 18,
    "title": "dolorum est consequatur ea mollitia in culpa",
    "completed": false
}
```

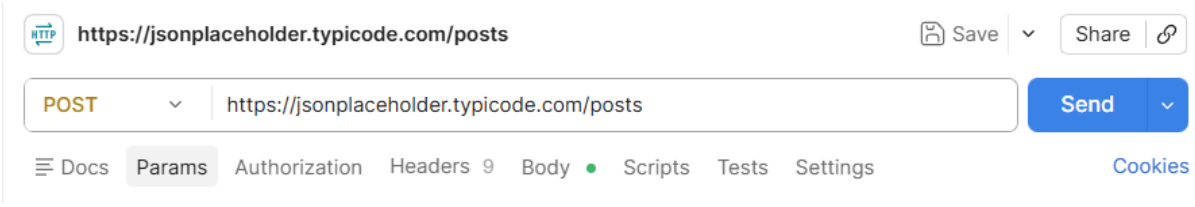

d) In one sentence, explain what changed in the results compared to not using any params.

Using query parameters filters the results, so only records matching the specified conditions are returned instead of all records.

TASK 3 Sending Data With Headers & a Body

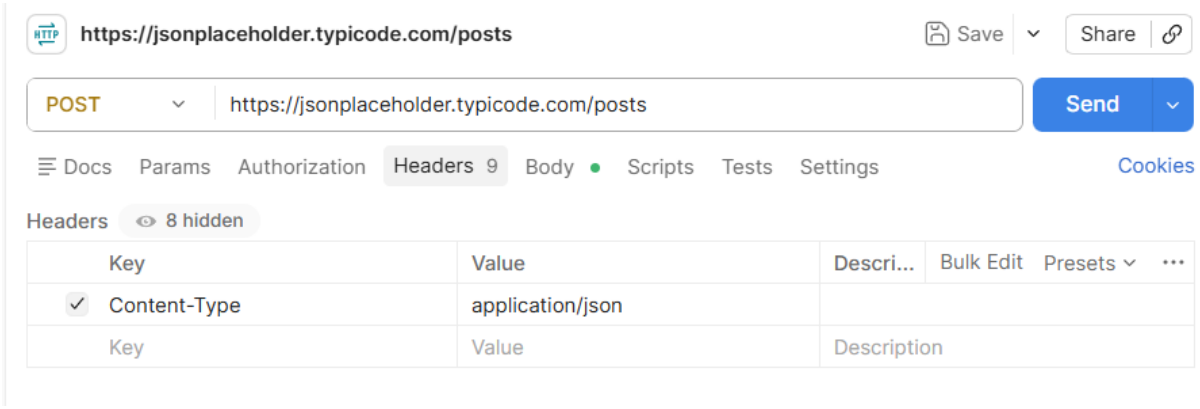
a) Send a POST request to /posts.

Output:



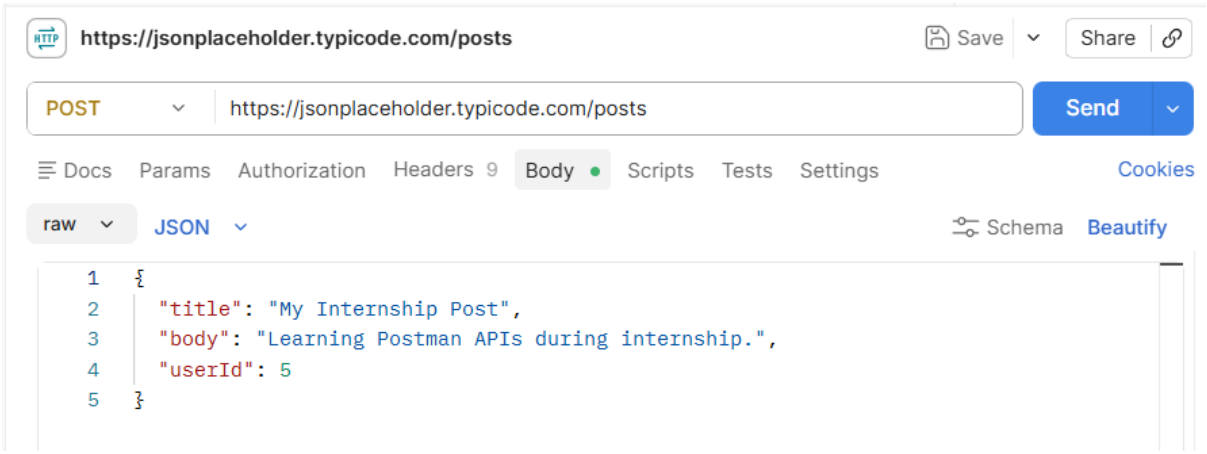
b) In the Headers tab, add: Content-Type: application/json

Output:



c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

Output:



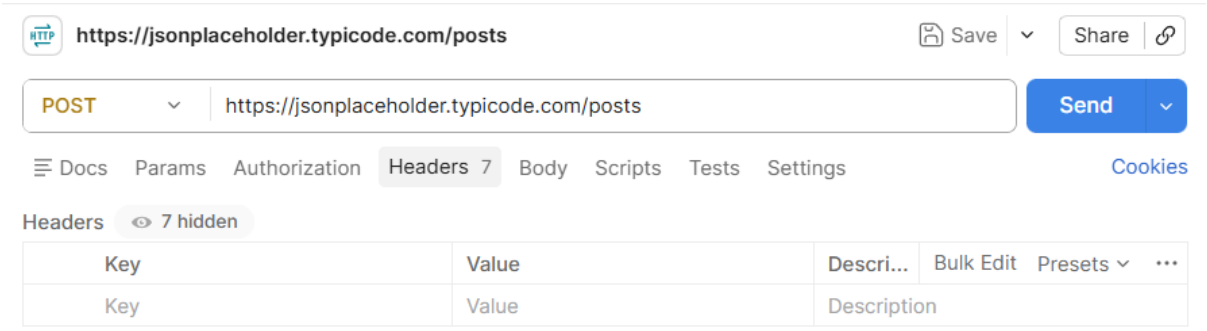
d) Record the status code and the new id the server assigned.

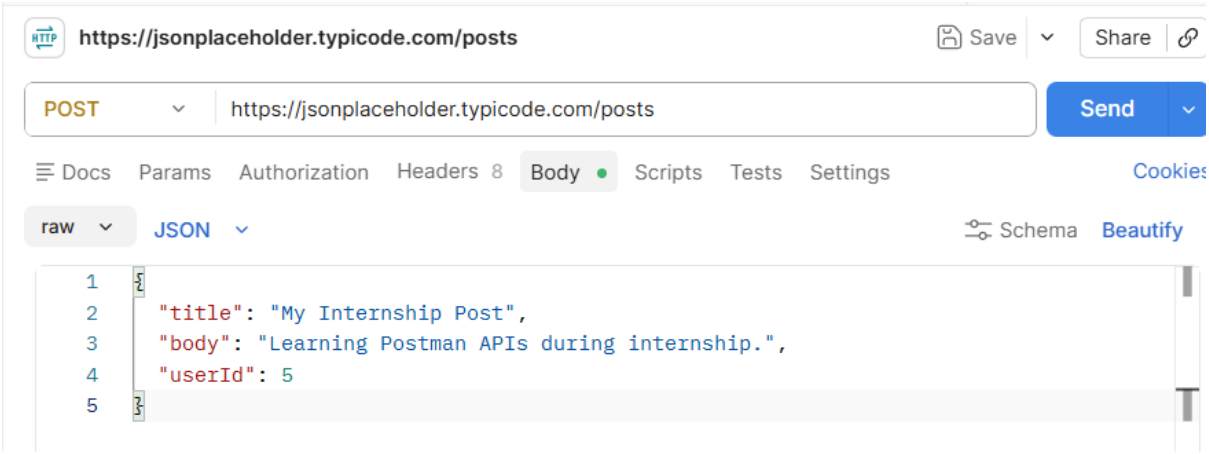
Output:

TASK 4 Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.

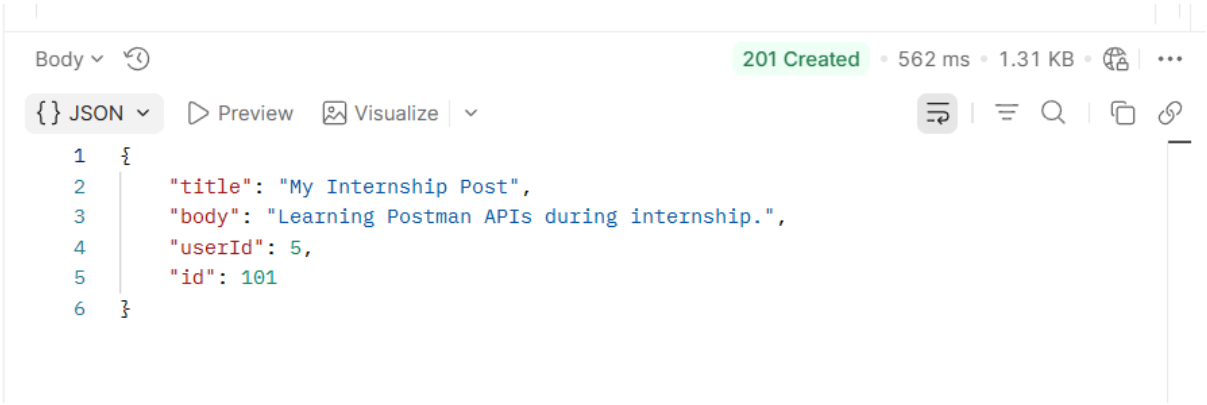
Output:





b) Compare the response to Task 3 — is it different? Record what you observe.

Output:



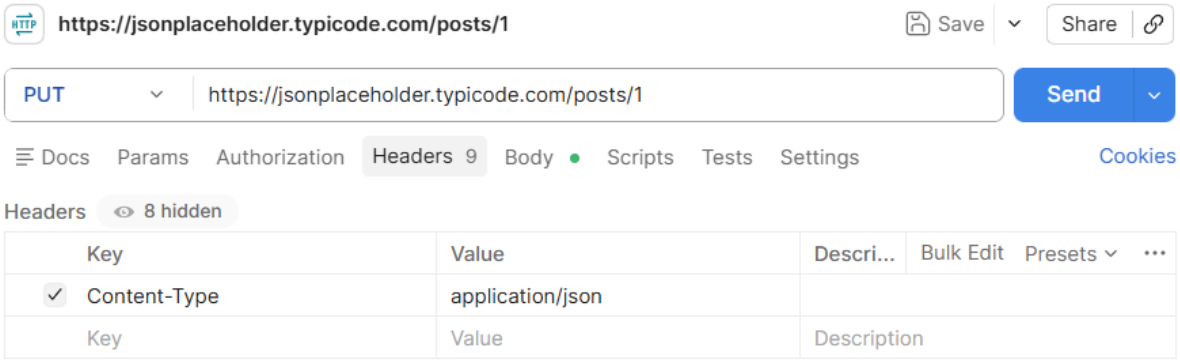
The response was almost the same as Task 3. JSONPlaceholder accepted the request even without the Content-Type header because it is a practice API and is designed to be flexible.

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

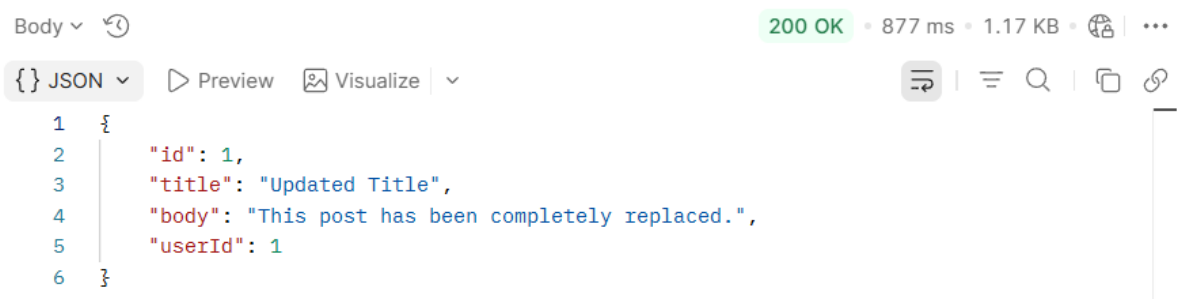
Headers tell the server important information about the request, such as the format of the data being sent. The Content-Type: application/json header lets the server know that the request body contains JSON. Although JSONPlaceholder accepts requests without this header, most real-world APIs require it to correctly process the request.

TASK 5 PUT vs. PATCH

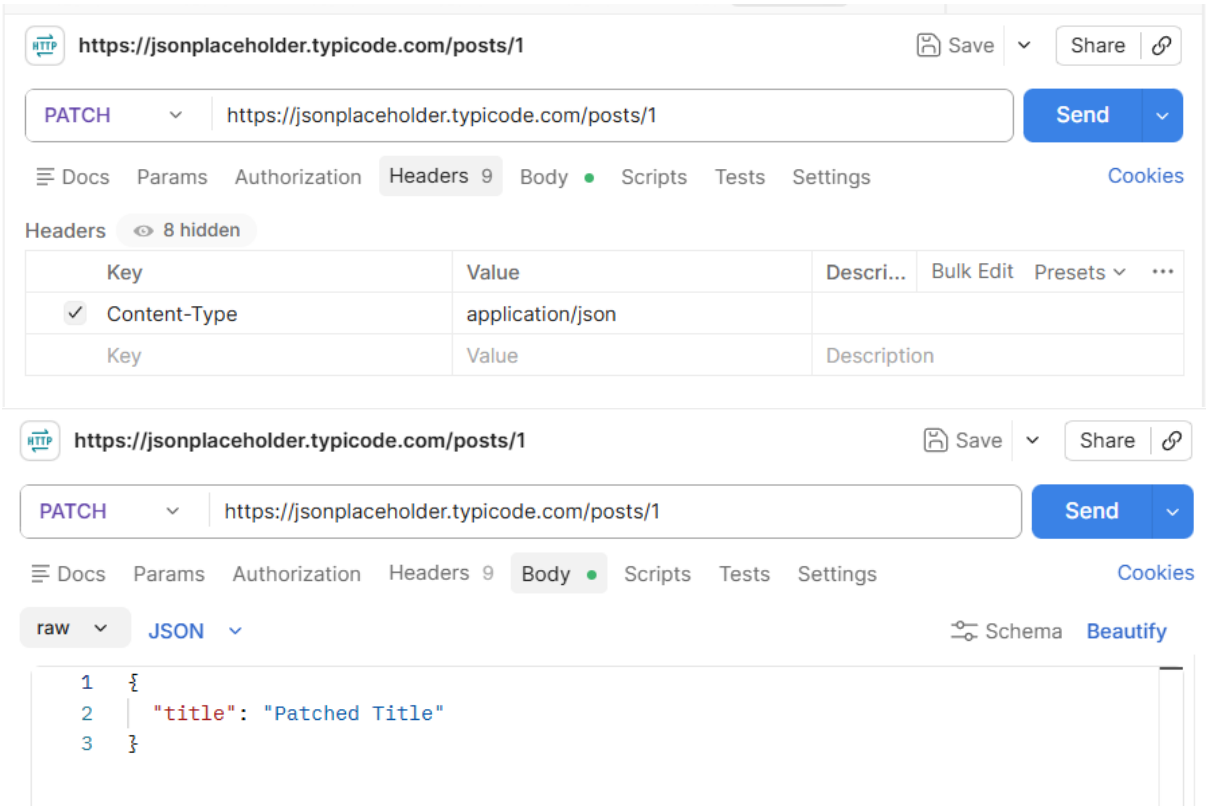
a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).



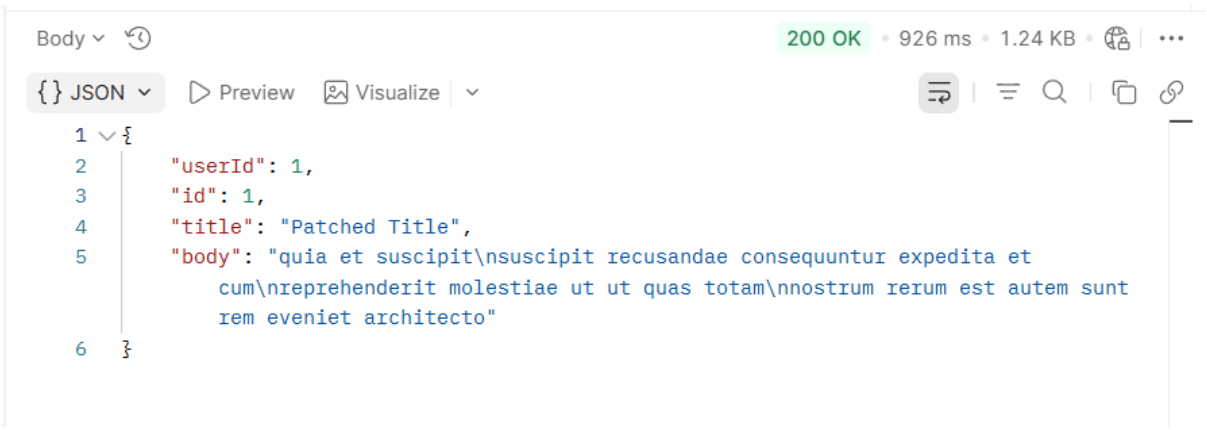
Output:



b) Send a PATCH request to the same /posts/1 that changes only the title.



Output:



c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

A PUT request replaces the entire resource, while a PATCH request updates only the specified field(s) without replacing the whole resource.

Final Challenge — A Realistic Request Flow

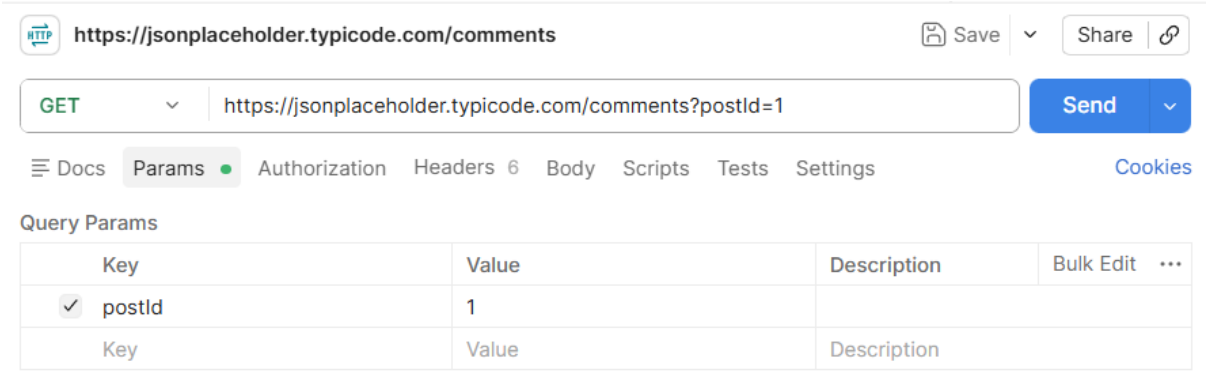
Concepts used: *this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.*

TASK 6 **The Mini Feedback System**

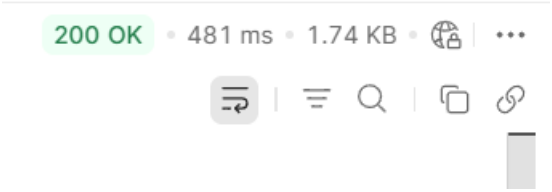
Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

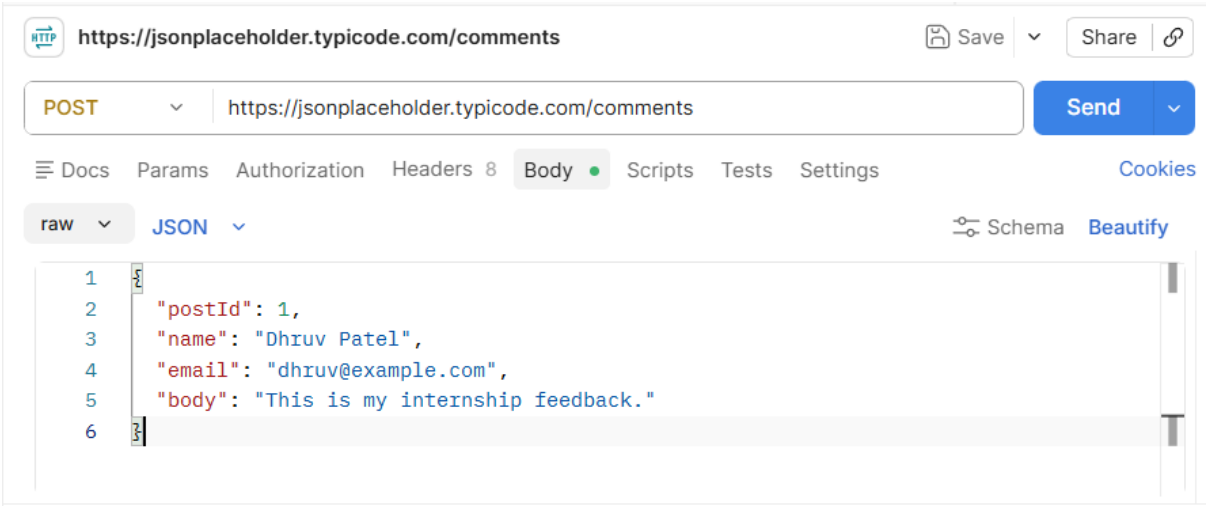
a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.



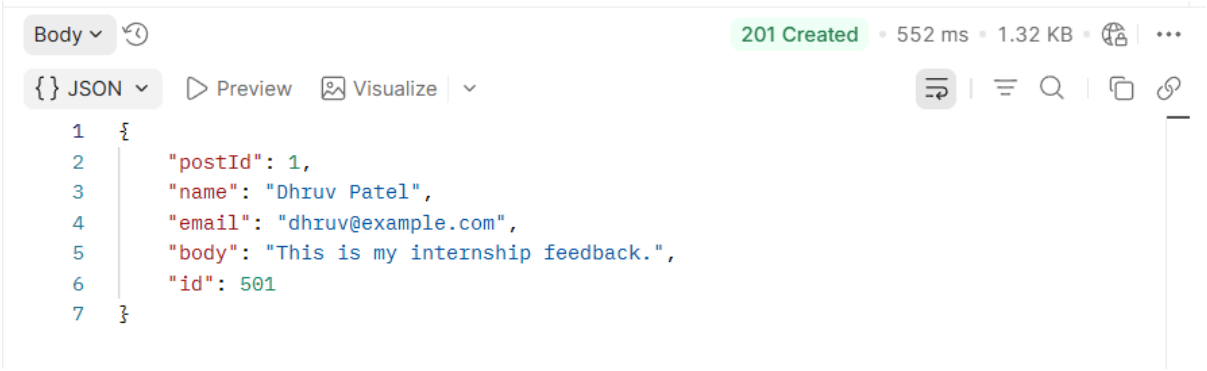
Output:



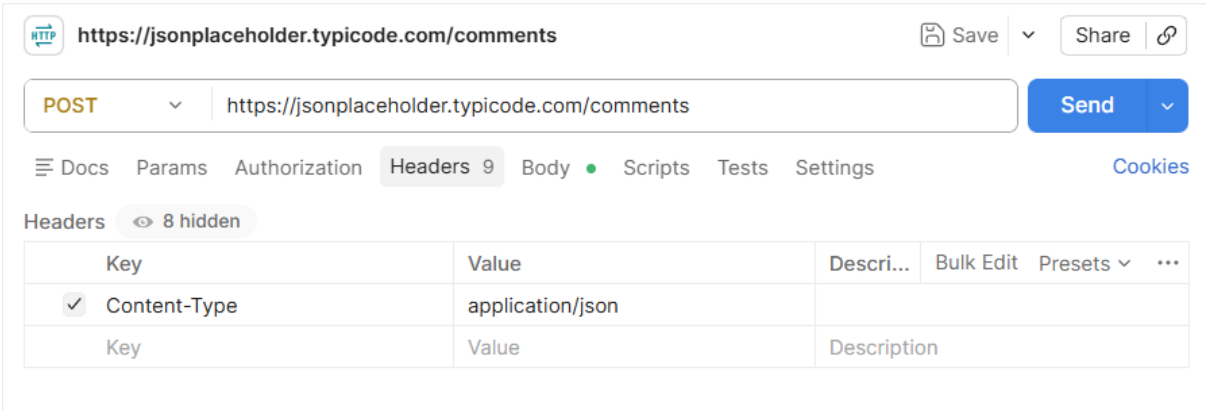
b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

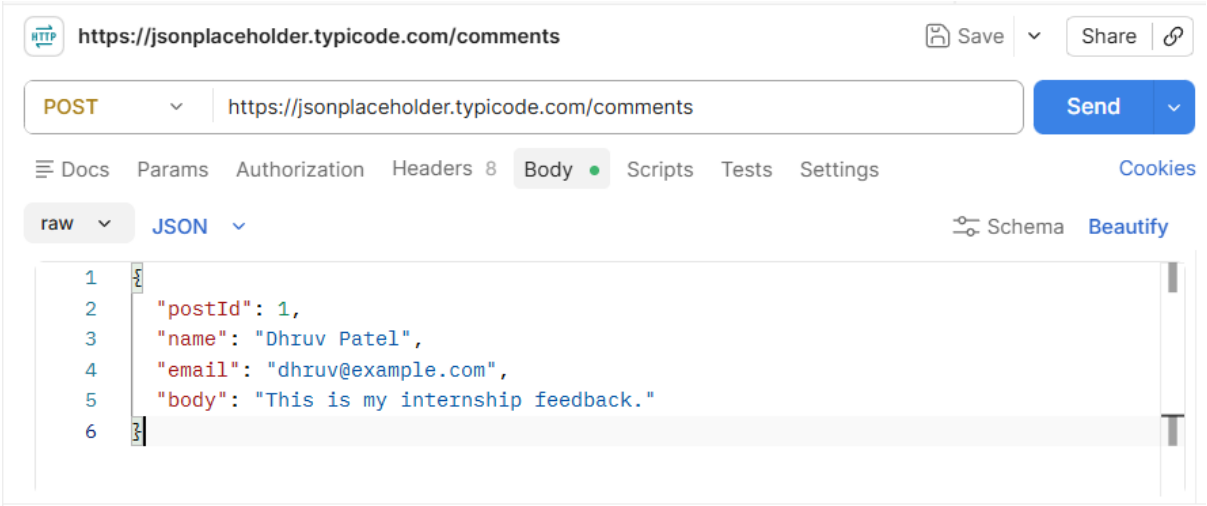


Output:

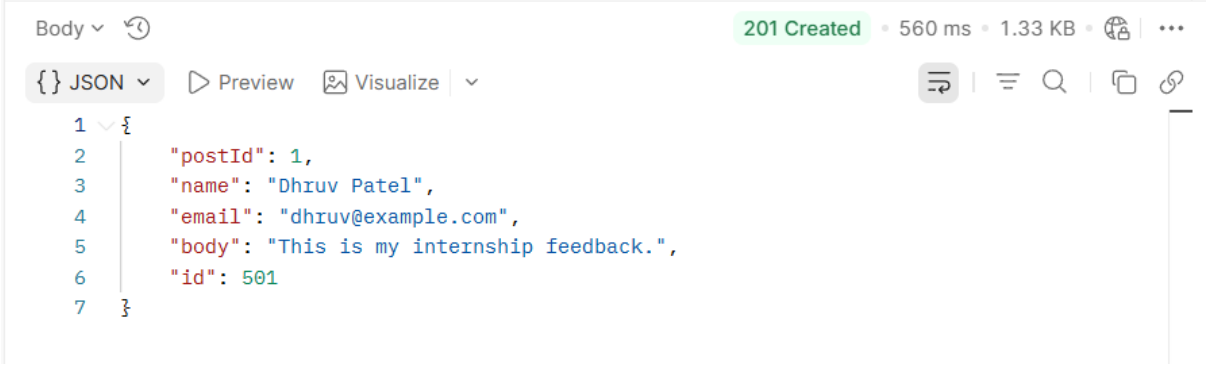


c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

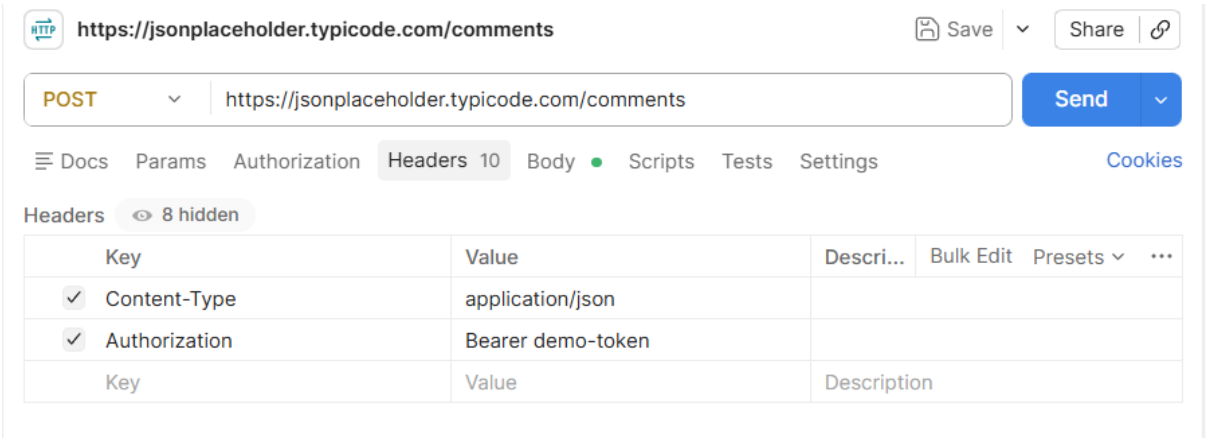


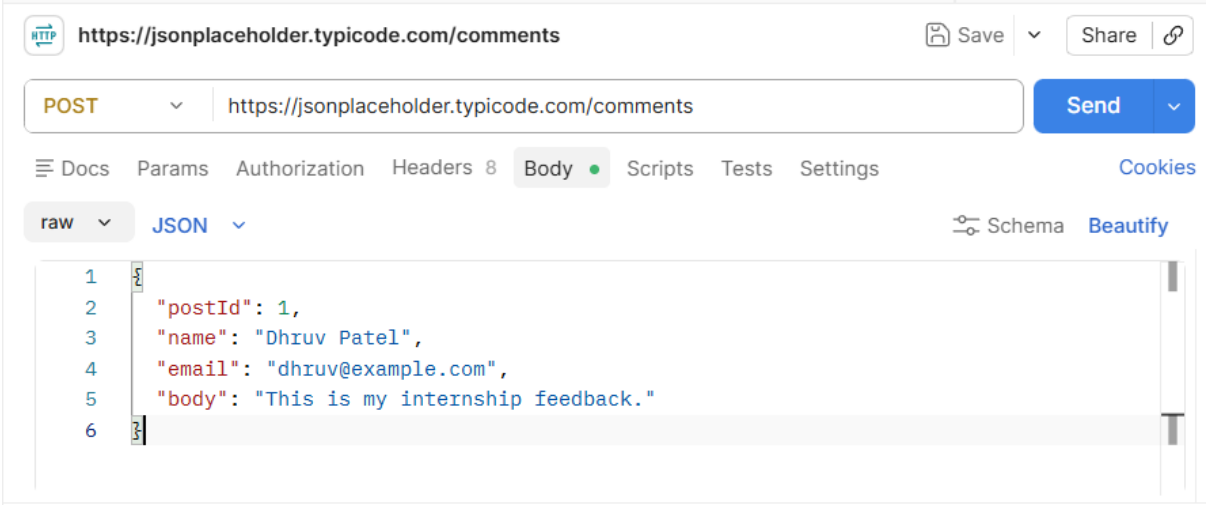


Output:

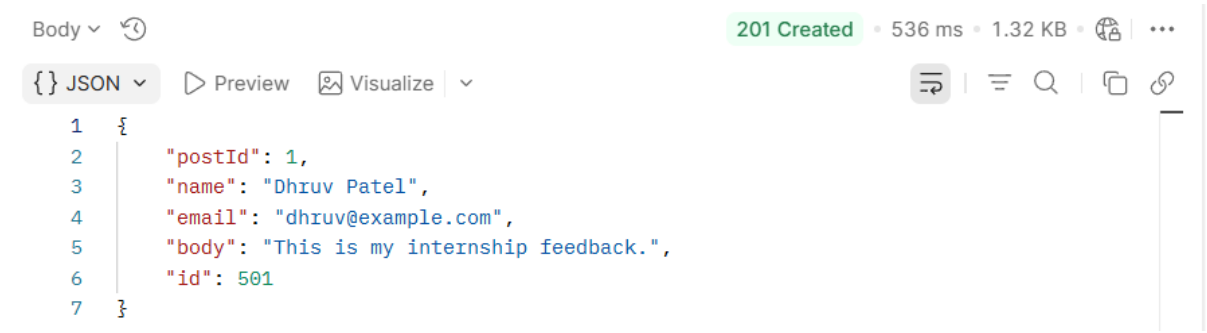


d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.





Output:



e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
(a)	GET /comments?postId=1	200 OK
(b)	POST /comments (No Headers)	201 Created
(c)	POST /comments (Content-Type Header)	201 Created
(d)	POST /comments (Content-Type + Authorization Header)	201 Created

f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

In a real API, the Authorization header would be checked to verify the user's identity before allowing access to protected resources. If the token were missing or invalid, the server would return an error such as **401 Unauthorized** or **403 Forbidden**. JSONPlaceholder ignores the header because it is only a practice API and does not perform authentication.

This helps protect sensitive data and ensures that only authorized users can access or modify protected resources.